FINAL PROJECT REPORT

C PROGRAMMING IMPLEMENTATION OF A SCIENTIFIC CALCULATOR

Team Members: PRITHU (2621066042)
TAHMID (2621591042)
RUMI (2623446042)
ARIF (2622986042)

Department: *Department of Electrical and Computer Engineering (ECE)*

Institution: North South University

Course: CSE115 Programming Language I

Overview—> This report presents the implementation and system structure of a console-based Scientific Calculator developed entirely in the C programming language . The application coordinates core procedural C concepts including user-defined data types, two-dimensional array matrices, fixed-dimension vectors, trigonometric transformations, logarithmic routines, base conversion algorithms, combinatorics, and dynamic loop control to prevent unexpected terminal exit. Program stability is maintained through dedicated buffer-clearing pause routines and conditional state updates. The system was organized into logical developer responsibilities and tested across mathematical calculations. The results demonstrate a modular, lightweight single-file console program designed for 1st Semester academic evaluation.

I. INTRODUCTION

A. Background

Developing a terminal-based scientific calculator requires combining linear algebra, floating-point arithmetic, unit conversions, and algebraic computations into a structured command-line interface. Rather than relying on multi-file dependencies, this project implements all mathematical modules and structural operations within a single source file environment . The system incorporates explicit header inclusion (`stdio.h`, `math.h`, `stdlib.h`, `string.h`, `ctype.h`) alongside precise user-defined data types and constant definitions to manage multi-dimensional mathematical structures.

B. Problem Statement

Implementing an interactive multi-functional calculator in standard C presents several functional and operational challenges:

- Managing recurring menu loops and terminal displays without forcing the user to restart the executable after completing a calculation.
-
- Supporting matrix and vector algebra using array bounds without exceeding memory limits.
-
- Parsing mathematical constraints such as undefined factorial inputs ($n < 0$) or invalid non-quadratic polynomial conditions ($a = 0$).
-
- Structuring diverse mathematical responsibilities (linear algebra, arithmetic, logarithms, base conversions, trigonometry, unit conversions, combinatorics) within a single codebase.
-

C. Objectives

1. Design and construct a console-based Scientific Calculator program in C .
2.
3. Define custom data types (typedef) and mathematical constants (PI, conversion factors) for clean code maintenance.
4.
5. Implement procedural routines for matrix operations (addition, subtraction, multiplication, recursive determinant, inverse) and 3D vector operations (addition, subtraction, dot product, cross product).
6.
7. Provide numeric conversion routines across decimal, binary, and hexadecimal representations, alongside logarithmic, trigonometric, and combinatorics modules.
8.
9. Ensure controlled program execution using state variables (rerun_program) and screen control utilities.
10.

II. SYSTEM CONSTANTS AND DATA TYPES

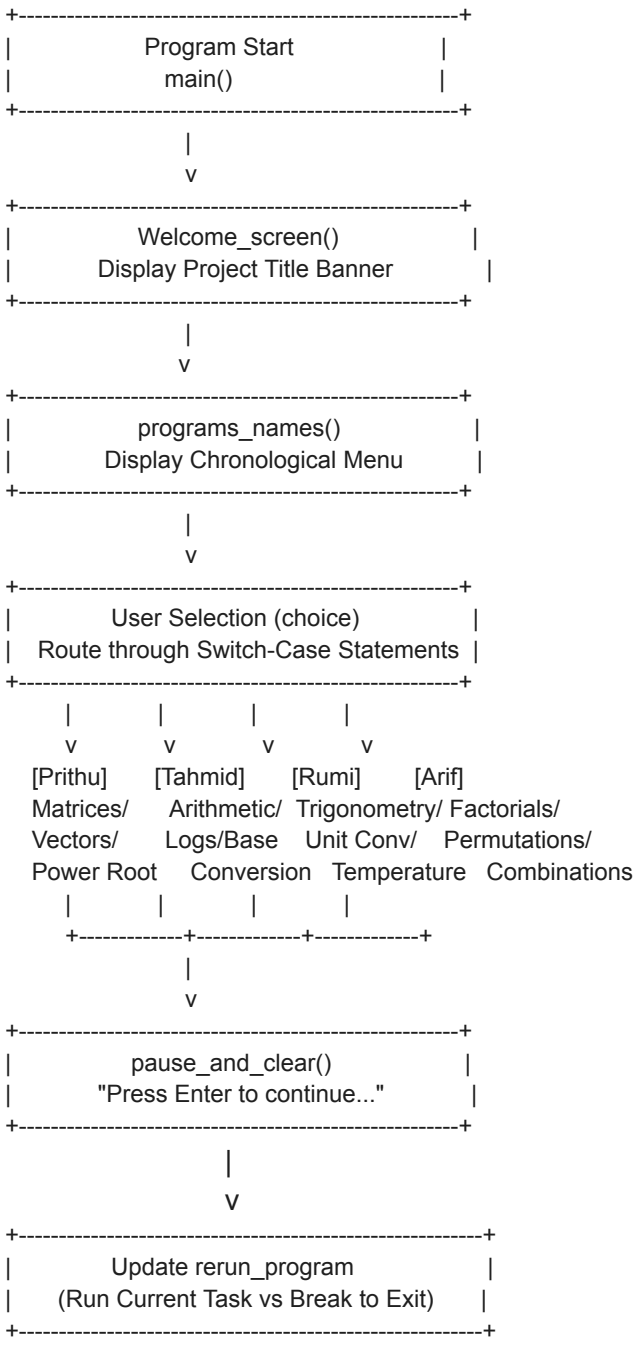
The application defines dynamic mathematical boundaries and unit conversion constants directly at the top of the program file to maintain continuous precision.

C

```c
#define max_matrix_size 10 // setting the maximum size that a matrix can have
#define max_vector_size 3  // setting maximum dimensions a vector can have

// creating new data types for matrix and vector structures
typedef double
typedef double vector

const double        3.14159265

/* constants for unit_conversion */
// distance
const double KM_TO_MILE =0.621371
const double METER_TO_FEET = 3.28084
const double CM_TO_INCH = 0.393701
const double METER_TO_YARD = 1.09361
const double KM_TO_NAUTICAL_MILE = 0.539957

// weight
const double KG_TO_POUND = 2.20462
const double GRAM_TO_OUNCE = 0.035274

// liquid
const double LITER_TO_GALLON = 0.264172
```

III. SYSTEM ARCHITECTURE AND FLOW

The calculator utilizes global execution control variables (`choice`, `rerun_program`) to maintain the application session within a continuous `while` loop.



IV. WORK DISTRIBUTION BY TEAM MEMBERS

The development tasks were distributed among the four team members based on functional sub-programs[cite: 1]:

Table I. Member Work Division and Functional Allocation

Team Member	Allocated Sub-Programs and Functional Prototypes
Prithu	Power & root operations (power_root), Matrix algebra (matrices, matrix_addition, matrix_subtraction, matrix_multiplication, matrix_determinant, calculate_determinant, get_submatrix, matrix_inverse), Vector operations (vectors, vector_addition, vector_subtraction, vector_dot, vector_cross)[cite: 1].

Tahmid	Basic arithmetic operations (arithmetic_operations, addition, subtraction, multiplication, division, modulus), Logarithmic operations (logarithms, natural_log, common_log, log_base_n), Base conversions (base_conversion, decimal_to_binary, binary_to_decimal, decimal_to_hexadecimal, hexadecimal_to_decimal)
Rumi	Trigonometric routines (trigonometry, standard/inverse calculations for sin, cos, tan, sec, cosec, cot, arcsin, arccos, arctan, arcsec, arccosec, arccot), Unit conversions (unit_conversion, distance, weight, liquid, temperature routines)
Arif	Combinatorics & Algebra (factorials, permutations, combinations, polynomial_roots_solving, polynomiyal_roots).

V. TESTING AND VALIDATION LOGIC

Table II. Mathematical Validation and Boundary Rules

Functional Component	Tested Input Scenario	Expected Internal Logic / Output
Factorial Module	Negative Integer ($n =- 5$)	Returns -1 (undefined factorial state)[cite: 1].
Permutation / Combination	Invalid Bounds ($r > n$)	Returns -1 (out of range condition)[cite: 1].
Polynomial Roots	Non-Quadratic Input ($a = 0$)	Displays error message: 'a' cannot be 0[cite: 1].

Vector Module	3D Space Elements	Maps components across i, j, and k spatial dimensions[cite: 1].
Base Conversion	String/Array Parsing	Converts string hex/binary indices into numeric output values[cite: 1].

VI. CONCLUSION

The Scientific Calculator project demonstrates the implementation of modular mathematical functions consolidated within a single C program source file (main.c). By incorporating custom dynamic array definitions, linear conversion factors, trigonometric conversion formulas, and combinatorics safeguards, the application provides an interactive terminal calculator suitable for 1st Semester academic evaluation. The structured division across team members organized module construction and accurate function mapping